

TRABAJO PRÁCTICO

“RETRASO LOGÍSTICO”

EQUIPO: DATA ROSE

Integrantes: Pilar Ospital, Chiara Behar, Felicitas Lacentre,
Valentina Montechi y María Emilia Rizzato

Informe - Trabajo Práctico “Retraso Logística”

Definición del Problema de Negocio

La empresa de logística necesita anticipar qué envíos tienen alta probabilidad de llegar con retraso antes de que el problema ocurra. Actualmente, los retrasos se detectan de manera reactiva, es decir, cuando el cliente ya sufrió el inconveniente. Por lo que la empresa pierde credibilidad y debe asumir costos de compensación.

Problema central: Predecir si un envío llegará con retraso (variable target: entrega_demorada = 1), o a tiempo (entrega_demorada = 0), utilizando información disponible en el momento de despacho del paquete.

Quién usa el modelo y para qué

El área de Operaciones y el equipo de Atención al Cliente son los usuarios principales del modelo. Lo utilizarían de forma diaria para:

- Priorizar el seguimiento activo de envíos con alto riesgo de retraso.
- Alertar al cliente de forma proactiva antes de que el retraso ocurra.
- Reasignar recursos logísticos (como rutas alternativas) en los paquetes de mayor riesgo.

Error más costoso: Falso Negativo vs Falso Positivo

Falso Negativo: El modelo predice entrega_demorada = 0, cuando en realidad entrega_demorada = 1. Predecir 'llega a tiempo' cuando en realidad llega tarde.

Consecuencias: cliente insatisfecho, compensaciones económicas, daño reputacional.

Falso Positivo: El modelo predice entrega_demorada = 1, cuando en realidad entrega_demorada = 0. Predecir 'va a retrasarse' cuando en realidad llega a tiempo.

Consecuencias: Recursos de seguimiento utilizados innecesariamente.

Conclusión: Se prioriza maximizar el Recall (minimizar falsos negativos), aunque implique aceptar más falsas alarmas, dado que los daños reputacionales y la pérdida de clientes representan un problema mayor para la empresa logística.

Descripción del Dataset

El dataset brindado es retraso_logistica.csv. El cual fue previamente limpiado en Google Colab mediante la librería Pandas. La limpieza se concentró en la variable valor_declarado_ars, cuyos valores incluían caracteres de texto y símbolos monetarios (ARS, \$ y espacios). Estos elementos fueron eliminados y la variable fue convertida a formato numérico, permitiendo su correcta utilización en el análisis posterior. Finalmente, el archivo procesado fue exportado en formato CSV. (Anexo 1)

Luego, se cargó en Orange mediante el widget File. A continuación se describe su estructura general:

Variable target:

- entrega_demorada: Indica si el paquete sufrirá un retraso (1 = demorado) o si llegará según lo previsto (0 = A tiempo).

Variable de identificación:

- id_pedido: El número identificador único de tracking o pedido.

Variables numéricas:

- peso_paquete_kg: Representa una medida física que puede tomar cualquier valor real con decimales.
- volumen_paquete: El tamaño de la caja expresado en centímetros cúbicos.
- distancia_km: Los kilómetros de distancia que existen entre el almacén de origen y el destino final.
- intentos_previos: La cantidad de visitas a domicilio fallidas acumuladas debido a que no se encontraba nadie en el lugar.
- valor_declarado_ars: Costo del producto asegurado.

Variables categóricas:

- fecha_despacho: La fecha y/o momento en que el paquete salió del centro de distribución.
- codigo_postal_dest: Código Postal Argentino.
- provincia_destino: La provincia de Argentina donde se debe realizar la entrega.
- categoría_producto: El rubro o sector al que pertenece el artículo comprado.
- proveedor_logistico: La empresa transportista asignada para realizar el envío.
- clima_despacho: El pronóstico del tiempo meteorológico registrado en el día de la salida del paquete.

- `es_vispera_feriado`: Variable que indica si el día siguiente o subsiguiente es un feriado nacional.
- `tipo_envio`: La modalidad de la entrega contratada (Normal, Express o Same-Day).
- `etiqueta_fragil`: Indica si el paquete requiere de una manipulación o cuidado especial durante su traslado.

Python Script de limpieza

El pipeline incluye un widget Python Script al inicio (conectado al File) que realiza transformaciones que Orange no puede hacer de forma nativa. Las transformaciones realizadas incluyen:

- Estandarización de nombres de columnas (eliminar espacios, caracteres especiales).
- Conversión de tipos de datos incorrectos (fechas leídas como texto, por ejemplo).
- Corrección de datos (como códigos postales con partes faltantes, valores nulos o faltantes)

Tratamiento de valores nulos

Se aplicó una estrategia diferenciada según el tipo de variable:

- Variables numéricas continuas: Se reemplazó el valor faltante por la media. (Ej. `Distancia_km`, y `Peso_paquete`)
- Variables categóricas: Se reemplazó el valor faltante por la moda (siempre respetando criterios anteriores, por ejemplo en caso de código postal faltante, se reemplazó por la moda, de la “provincia” correspondiente a cada registro). Utilizando un widget de Python.

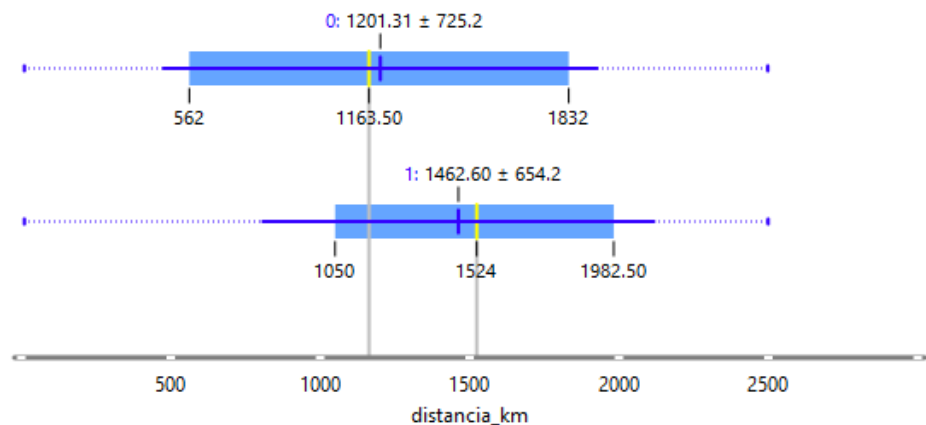
Análisis Exploratorio de Datos (EDA)

El EDA se realizó en Orange utilizando los widgets: Distributions, Box Plot, Scatter Plot, Mosaic Display y Data Table. A continuación se presentan las hipótesis formuladas y los hallazgos obtenidos.

Hipótesis planteadas

H1 — Distancia del envío y demora | `distancia_km`

Hipótesis 1: a mayor distancia, mayor probabilidad de demora.



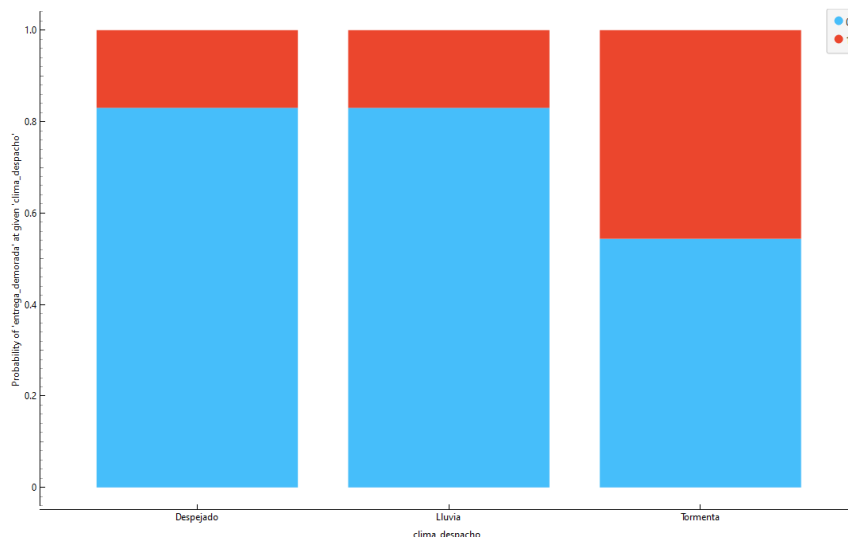
Los envíos demorados recorren en promedio 261 km más (media 1.462 km vs 1.201 km). El Q1 del grupo demorado (1.050 km) casi duplica al del grupo a tiempo (562 km), lo que refuerza la relación. Existe solapamiento entre ambas distribuciones, por lo que la distancia sola no predice el retraso — debe combinarse con otras variables.

Conclusión: variable recomendada para incluir en el modelo. Se sugiere crear una categoría “Tipo distancia” para capturar el efecto de forma no lineal.

Para esto se creó una nueva variable binaria “Tipo Distancia” de manera que 1 representa las distancias mayores a 1300 kilómetros (Distancia Larga) y 0 las menores a 1300 km (Distancia Corta). La distancia de 1300 no es arbitraria, sino que representa aproximadamente la mediana global del dataset, que cae entre la mediana del grupo a tiempo (1.163 km) y la mediana del grupo demorado (1.524 km).

H2 — Clima al momento del despacho y demora

Hipótesis 2: el clima adverso (como lluvias o tormentas) al momento del despacho aumenta la tasa de demora.

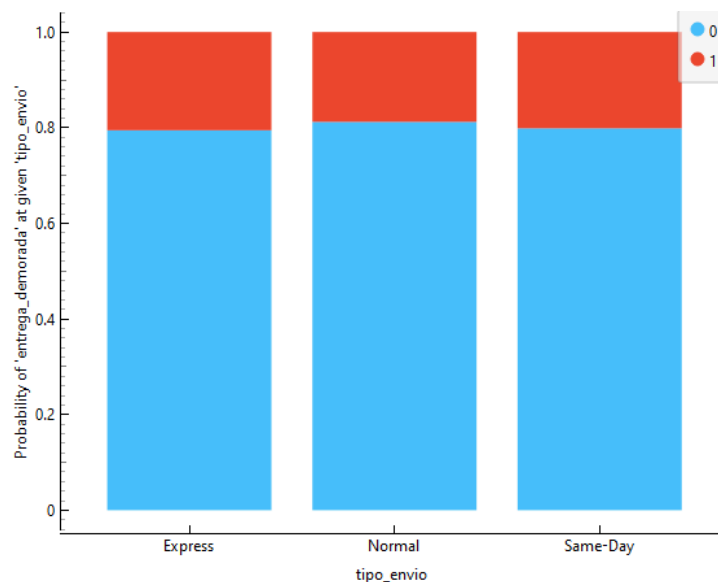


Los datos muestran una diferencia drástica según la condición climática. Con cielo despejado y con lluvia, la tasa de demora ronda el 17–18%, prácticamente igual. Pero bajo condiciones de tormenta, la tasa aumenta a aproximadamente 45%, casi el triple.

Conclusión de negocio: la lluvia por sí sola no genera demoras adicionales. La tormenta, en cambio, es un predictor fuerte e independiente.

H3 — Tipo de envío y demora | tipo_envio

Hipótesis 3: el tipo de envío (Express / Normal / Same-Day) influye en la probabilidad de demora.

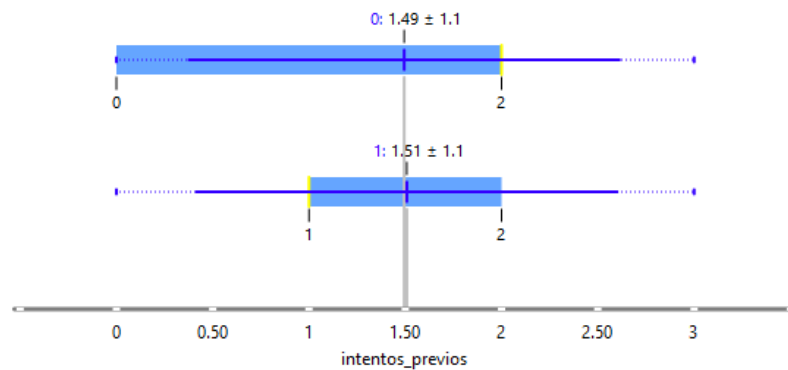


Los tres tipos de envío muestran una proporción de demoras prácticamente idéntica (~20%). Las barras de Express, Normal y Same-Day son visualmente indistinguibles en el gráfico de distribución.

Conclusión de negocio: el tipo de envío no aporta poder predictivo. Se recomienda no incluirlo como variable en el modelo para evitar ruido innecesario.

H4 — Intentos previos de entrega y demora | intentos_previos

Hipótesis 4: mayor cantidad de intentos previos fallidos se asocia con mayor probabilidad de demora.



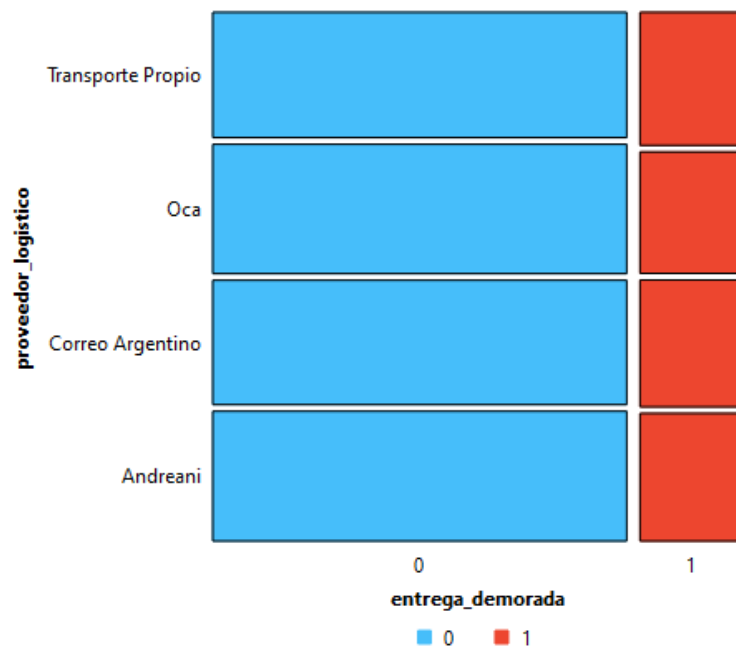
Las medias son prácticamente iguales en ambos grupos (1.49 para entregas a tiempo vs. 1.51 para entregas demoradas). El test de Student confirma la ausencia de diferencia real:

- Media grupo 0 (a tiempo): 1.49 ± 1.1
- Media grupo 1 (demorado): 1.51 ± 1.1
- Student's t: 0.497 — $p = 0.619$ (N = 10.000)

Conclusión de negocio: con $p = 0.619$, muy por encima del umbral de 0.05, no hay evidencia de que los intentos previos expliquen la demora. Se sugiere descartar la variable para el modelo.

H5 — Proveedor logístico y demora | proveedor_logistico

Hipótesis 5: existen diferencias en la tasa de demora según el proveedor logístico utilizado.

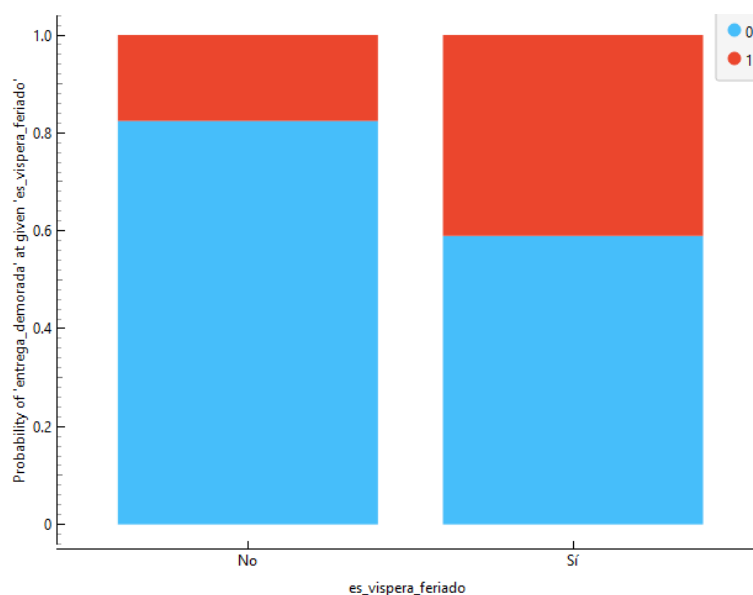


El gráfico Mosaic Display muestra barras de tamaño casi idéntico para los cuatro proveedores. Ninguno presenta una tasa de demora notablemente mayor o menor que los demás.

Conclusión de negocio: el proveedor logístico no es un factor diferenciador de retrasos en este dataset. Variable no recomendada para el modelo.

H6 — Víspera de feriado y demora | es_vispera_feriado

Hipótesis 6: los envíos despachados en víspera de feriado tienen mayor probabilidad de demorarse.



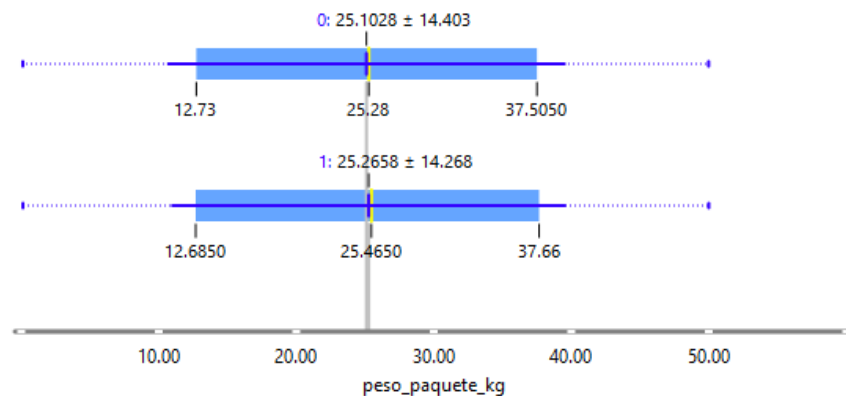
La diferencia es la más marcada de todas las hipótesis analizadas:

- Víspera de feriado = 0: tasa de demora ~18%
- Víspera de feriado = 1: tasa de demora ~41% — más del doble

Conclusión de negocio: despachar en víspera de feriado aumenta drásticamente el riesgo de demora, esto puede atribuirse a la sobrecarga operativa en los centros de distribución. Es la variable más predictora identificada en el EDA, es por esto que se incluye como prioridad en el modelo.

H7 — Peso del paquete y demora | peso_paquete_kg

Hipótesis 7: paquetes más pesados se demoran más por requerir logística especial.



Las distribuciones son prácticamente idénticas en ambos grupos:

- Media grupo 0 (a tiempo): 25.10 ± 14.40 kg — IQR: 12.73 a 37.51 kg
- Media grupo 1 (demorado): 25.27 ± 14.27 kg — IQR: 12.69 a 37.66 kg
- Student's t: 0.455 — $p = 0.649$ (N = 10.000)

Conclusión de negocio: con $p = 0.649$, el peso del paquete no predice la demora. Contrariamente a lo esperado, la logística de paquetes pesados no genera mayor retraso en este dataset. Variable descartada.

Preprocesamiento y Selección de Variables

En el pipeline de Orange se implementaron múltiples pasos de preprocesamiento. A continuación se documentan las decisiones tomadas y su justificación de negocio.

Detección y tratamiento de outliers

Se utilizó el widget Outliers para detectar observaciones anómalas. El flujo en Orange fue:

- File → Python Script (limpieza inicial) explicada anteriormente → Data Table → Outliers

Los outliers detectados (el 10% de los datos más extremos/atípicos, utilizando el método de “Isolation Forest” ya que es el más sencillo para modelos con datasets grandes), se separaron en dos ramas: Inliers (datos limpios) y Outliers, luego se tomaron solo los inliers para continuar.

Tratamiento de variables categóricas

El pipeline incluye el widget de “Continuize”, lo que genera que las variables categóricas fueran convertidas a formato numérico para su posterior utilización en los modelos que lo requieren. Se aplicó one-hot encoding para estas variables.

Feature Engineering - Variables derivadas

Se creó una nueva variable para capturar relaciones que las variables originales no expresan directamente. Esta variable es la de distancia larga (variable binaria) la cual fue creada a través del widget “Fórmula” (tipo distancia = 1, si la distancia es > 1300) con el objetivo de evaluar si las distancias categorizadas como “largas” concentran mayor proporción de entregas retrasadas.

Variable nueva	Descripción	Justificación
Tipo distancia	1 si distancia > 1300 km, 0 si no	Evaluar la posibilidad de que la distancia recorrida afecte la probabilidad de que el pedido llegue demorado (de forma más concentrada y eficiente que “distancia_km”)

Selección de columnas

El widget Select Columns del pipeline se utilizó para:

- Eliminar columnas de identificación (como ID de envío) que no tienen poder predictivo.

- Confirmar que la variable target queda correctamente designada como 'target' en Orange.
- Eliminar las variables que no tienen verdadero valor predictivo (según el EDA realizado) de manera que se disminuya el “ruido” en el modelo. Las variables que se mantuvieron para realizar el modelo son:
 - clima_despacho=Lluvia
 - clima_despacho=Tormenta
 - clima_despacho=Despejado
 - es_vispera_feriado=Sí
 - es_vispera_feriado=No
 - Tipo distancia=0
 - Tipo distancia=1

Creación de datos

Para mejorar la calidad de las predicciones, se procedió a crear mediante un script de Python, instancias (o registros) adicionales, con la variable entrega demorada = 1, esto se realizó para “balancear” la muestra y que durante la etapa de “training “ el modelo tenga contacto con suficientes instancias retrasadas. Se crearon instancias hasta llegar a una proporción del 65/35, de forma que el modelo conozca al menos 4001 instancias de pedidos entregados con demora. (El código puede verse en el anexo 3)

Para la creación de los datos nuevos, el modelo toma 2 registros al azar. Para las variables numéricas pondera cada registro con un alpha aleatorio, mientras que para las variables categóricas toma al azar uno de los dos valores. (Anexo 3)

Adicionalmente se agregó una columna que indica si el dato es nativo de la base de datos original o sintético (creado por código) para aportar trazabilidad.

Modelado

Se entrenaron tres algoritmos de clasificación, todos evaluados con el mismo split de datos para garantizar una comparación justa. El pipeline de Orange incluye los tres learners (Logistic regression, Random Forest y Naive Bayes) conectados al widget Test and Score.

Estructura del pipeline de modelado en Orange

El flujo implementado en el .ows sigue esta arquitectura:

- Data (pre-procesada) → Data Sampler → Training (80%) / Test (20%)

- Training → Logistic Regression + Random Forest + Naive Bayes
- Los tres modelos → Test and Score (Cross Validation K=10)
- Test → Predictions → Confusion Matrix + ROC Analysis

Se utilizó cross validation con k=10, ya que este divide el dataset en 10 partes iguales. Luego entrena y evalúa el modelo 10 veces, donde cada vuelta usa una parte distinta para testear y el resto para entrenar. Al final promedia los resultados de las 10 vueltas. Esto evita que el resultado dependa de "suerte" en cómo se dividieron los datos. Si solo se dividiera una vez, puede que por casualidad el test sea fácil o difícil. Con 10 folds el resultado es más representativo.

Modelo 1: Regresión Logística

Widget Orange: Logistic Regression. Es el modelo base e interpretable del análisis.

- Ventaja principal: permite entender directamente el peso de cada variable en la predicción (a través del Nomogram).
- Ideal para comunicar resultados al área de negocio: 'si el peso aumenta X kg, la probabilidad de retraso aumenta Y%.'

Modelo 2: Random Forest

Widget Orange: Random Forest. Es el modelo de mayor complejidad y generalmente el de mejor performance.

- Ventaja principal: captura relaciones no lineales y la interacción entre variables sin necesidad de transformaciones manuales.
- Maneja bien variables con diferentes escalas y categorías sin necesidad de normalización.

Modelo 3: Naive Bayes

Widget Orange: Naive Bayes. Es el tercer algoritmo de comparación.

- Ventaja: extremadamente rápido de entrenar y simple de interpretar. Buen punto de referencia.
- Supuesto: asume independencia entre variables predictoras.

Activación de Stratified en Cross Validation

Como el dataset presenta un desbalance de clase, se activó la opción 'Stratified' en Test and Score. Esto garantiza que cada fold del Cross Validation tenga la misma proporción de clases, evitando que el modelo aprenda de manera sesgada.

Evaluación del Modelo

Se evaluaron los tres modelos usando AUC, F1, Precision y Recall. Adicionalmente, se analizó la Confusion Matrix del modelo ganador para interpretar los errores en términos de negocio.

Tabla comparativa de modelos

La tabla a continuación muestra los resultados para la clase 1 (demorados), ya que son los que se propone detectar (minimizar los falsos negativos).

Evaluation results for target 1							
Model	AUC	CA	F1	Prec	Recall	MCC	
Logistic Regression	0.662	0.592	0.561	0.450	0.747	0.247	
Random Forest	0.661	0.592	0.561	0.450	0.746	0.246	
Naive Bayes	0.659	0.682	0.328	0.630	0.222	0.220	

Interpretación de métricas en contexto de negocio

Métrica	Que mide	Significado para el negocio
AUC	Capacidad general de separar las dos clases.	Si $AUC > 0.85$ el modelo puede distinguir bien los envíos en riesgo de los que llegan a tiempo.
Recall	De todos los envíos que realmente se retrasaron, cuántos detectó el modelo.	Si $Recall = 0.70$ significa que el modelo 'predice' el 70% de los retrasos antes de que ocurran.
Precisión	De todos los que el modelo marco como 'en riesgo', cuántos realmente se retrasaron.	Precisión baja significa muchas falsas alarmas: el equipo de operaciones revisará envíos que llegaron bien.
F1 Score	Balance entre Precisión y Recall en un único número.	Útil cuando no se puede priorizar sólo una de las dos métricas anteriores.

CA	El porcentaje de predicciones correctas sobre el total de casos	Fácil de entender, pero engañosa cuando las clases están desbalanceadas. Un modelo que siempre predice "NO" puede tener CA alta sin ser útil.
MCC	La calidad general de la clasificación considerando aciertos y errores de ambas clases. Va de -1 a +1.	Un valor cercano a 0 indica que el modelo no es mejor que el azar.

Dado que en el contexto del negocio el falso negativo representa un mayor costo que el falso positivo (un envío demorado no detectado implica un cliente insatisfecho y posibles compensaciones) se prioriza el modelo con mayor Recall para la clase 1 (demorado). En base a los resultados obtenidos, el modelo seleccionado es Logistic Regression, que alcanzó un Recall de 0.747, detectando correctamente el 74,7% de los envíos con demora.

Análisis de la Confusion Matrix

Se analizó la Confusion Matrix del modelo seleccionado como ganador (Regresión Logística). El flujo en Orange fue: Predictions → Confusion Matrix + ROC Analysis.

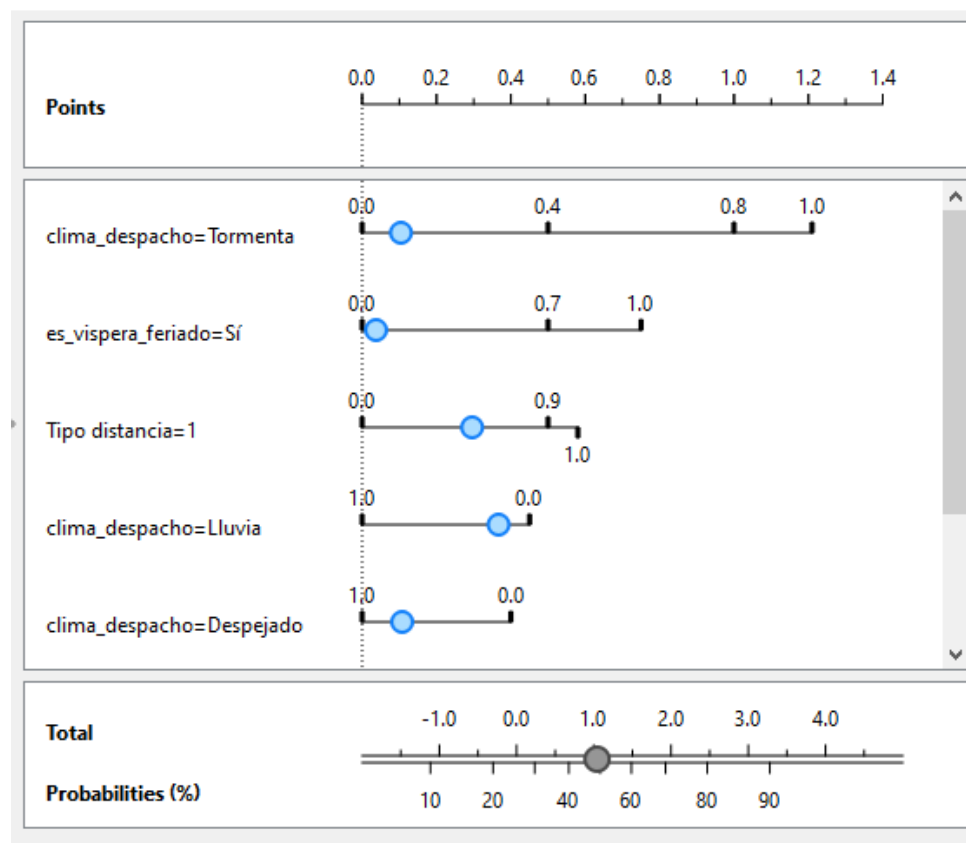
		Predicted		
		0	1	Σ
Actual	0	771	715	1486
	1	218	582	800
Σ		989	1297	2286

En el caso de Regresión Logística, el error más costoso representa 218 casos (las personas a las que no se les avisa del retraso de su paquete, pero este llega retrasado). Pero al utilizar el modelo, 582 de 800 sí recibirán una alerta previa (representando aproximadamente el 72,75% de los casos con retraso) respecto del 50% esperado si se hiciera al azar.

Quedarse con este modelo tiene un costo adicional, representado por 715 casos de “falsas alarmas” (aproximadamente un 48,1%) (casos que se notifican como retraso, pero llegan a tiempo). Sin embargo se elige este modelo, porque es el que minimiza el error más costoso para la empresa (FN).

Variables más importantes

Se utilizó el widget Nomogram (para Regresión Logística) para identificar las variables con mayor peso en la predicción de retrasos.



Para evaluar qué variables inciden en la predicción de retrasos, se analizó el nomogram del modelo configurado para la clase 1 (retraso). Este gráfico muestra el peso de cada variable sobre la probabilidad predicha. Los resultados indican que las condiciones climáticas adversas al momento del despacho especialmente tormenta y la proximidad a un día feriado son los factores que más aumentan la probabilidad de retraso.

Conclusión: el modelo aprendió que los factores externos y temporales (clima de tormenta y víspera de feriado) son los principales determinantes de una entrega demorada, por encima de las características físicas del paquete (como peso o volumen).

Recomendaciones concretas para el negocio

1. Revisión focalizada de envíos clasificados como retraso

Implementar una revisión diaria que separe automáticamente los envíos según la predicción del modelo: los clasificados como entrega_demorada = 1 deberían ingresar a un flujo de seguimiento activo, mientras que los clasificados como entrega_demorada = 0 siguen el proceso estándar. Esto concentra la atención operativa únicamente en los envíos que el modelo identifica como problemáticos, reduciendo la revisión manual del 100% de los casos a una fracción significativamente menor.

2. Intervención preventiva para envíos con predicción de retraso

Los envíos que el modelo clasifica como entrega_demorada= 1 deberían ser derivados automáticamente a un proveedor logístico prioritario o ruta alternativa en el momento del despacho, antes de que el retraso ocurra. Las variables que más pesan en esa clasificación son: tormenta, víspera de feriado y Tipo distancia pueden anticiparse con horas de antelación, lo que hace viable la intervención preventiva.

Condicionado a que el costo de la intervención preventiva sea menor al costo de la compensación al cliente por retraso.

3. Comunicación proactiva con el cliente

Para todos los envíos clasificados como retraso = 1, activar una notificación automática al cliente informando que el envío podría demorar más de lo esperado. Esto transforma un problema reactivo en una comunicación proactiva, mejorando la experiencia del cliente incluso cuando el retraso es inevitable.

Hallazgo sorprendente o contra-intuitivo

Si bien en el EDA, distancia_km resultó ser una variable poco significativa, al agruparse las distancias en dos subgrupos (distancia larga vs distancia corta) se encontró que esta variable sí tenía un impacto significativo en el posible retraso de un pedido (siendo la 3ra con más poder de predicción en el modelo).

Pedido del Profesor:

A pedido del profesor, luego de haber dado la presentación. Adjuntamos la foto del resultado de los modelos, con sus métricas asociadas (incluyendo las variables que habíamos ignorado por el EDA).

Test and Score - Orange

☒ Cross validation

Number of folds: 10

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 80 %

☒ Stratified

Evaluation results for target 1

Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.662	0.596	0.558	0.452	0.728	0.243
Random Forest	0.756	0.715	0.572	0.602	0.546	0.360
Naive Bayes	0.690	0.683	0.470	0.567	0.401	0.261

Como lo comentamos en la presentación oral, las métricas no mejoran e incluso algunas empeoran, aun incluyendo estas variables. Igualmente, no queríamos dejar de mostrarlo.

Anexos:

Códigos de Python

Anexo 1 (Limpieza valor_declarado_ars)

```
import pandas as pd
df = pd.read_excel("Retraso_Logistica.xlsx")

# Limpiar valor declarado
df["valor_declarado_ars"] = (
    df["valor_declarado_ars"]
    .astype(str)
    .str.replace("ARS", "", regex=False)
    .str.replace("$", "", regex=False)
    .str.replace(" ", "", regex=False)
)

df["valor_declarado_ars"] = pd.to_numeric(df["valor_declarado_ars"], errors="coerce")
df.to_csv("Retraso_Logistica_limpio.csv", index=False)

from google.colab import files
files.download("Retraso_Logistica_limpio.csv")
```

Anexo 2:

```
import re

from collections import Counter
out_data = in_data.copy()
# PASO 1 — Extraer los 4 dígitos del medio
for i, row in enumerate(out_data):
    valor = str(row["codigo_postal_dest"]).strip().upper()
    numeros = re.search(r'\d{4}', valor)
    out_data[i]["codigo_postal_dest"] = numeros.group() if numeros else ""
# PASO 2 — Construir moda por provincia (datos ya limpios)
provincia_a_codigos = {}
for row in out_data:
    provincia = str(row["provincia_destino"]).strip()
    codigo = str(row["codigo_postal_dest"]).strip()
    if codigo and codigo not in ("", "?", "None", "nan"):
```

```

provincia_a_codigos.setdefault(provincia, []).append(codigo)
provincia_a_moda = {
    prov: Counter(cods).most_common(1)[0][0]
    for prov, cods in provincia_a_codigos.items()
}
# PASO 3 — Rellenar vacíos con la moda de su provincia
for i, row in enumerate(out_data):
    codigo = str(row["codigo_postal_dest"]).strip()
    if codigo in ("", "?", "None", "nan"):
        provincia = str(row["provincia_destino"]).strip()
        out_data[i]["codigo_postal_dest"] = provincia_a_moda.get(provincia, "0000")
    print(f"✅ Listo. Filas procesadas: {len(out_data)}")
    print(f"📍 Provincias: {list(provincia_a_moda.keys())}")

```

Anexo 3:

```

vals = np.array([str(inst.get_class()) for inst in in_data])
idx_0 = np.where(vals == clase_mayoritaria)[0]
idx_1 = np.where(vals == clase_minoritaria)[0]
n_majority = len(idx_0)
n_minority = len(idx_1)
print(f"ORIGINAL → '{clase_mayoritaria}': {n_majority} | '{clase_minoritaria}': {n_minority} | Ratio: {n_majority/n_minority:.1f}:1")
TARGET_RATIO = 0.65
n_class1_target = int(n_majority * (1 - TARGET_RATIO) / TARGET_RATIO)
n_needed = max(0, n_class1_target - n_minority)
print(f"A generar: {n_needed} sintéticos de '{clase_minoritaria}'")
domain = in_data.domain
minority_table = in_data[idx_1]
X_min = minority_table.X
Y_min = minority_table.Y
continuous_mask = np.array([attr.is_continuous for attr in domain.attributes])
print(f"Numéricas: {[a.name for a in domain.attributes if a.is_continuous]}")
print(f"Católicas: {[a.name for a in domain.attributes if not a.is_continuous]}")
# Detectar columnas numéricas que deben ser enteros
# (las que en los datos originales no tienen decimales)
integer_mask = np.zeros(len(domain.attributes), dtype=bool)
for k, attr in enumerate(domain.attributes):

```

```

if attr.is_continuous:
col = X_min[:, k]
col_clean = col[~np.isnan(col)]
if np.all(col_clean == col_clean.astype(int)):
integer_mask[k] = True
print(f" → '{attr.name}' se redondeará a entero")
# Generar sintéticos
synthetic_X = []
meta_source_indices = []
for _ in range(n_needed):
i, j = np.random.choice(len(minority_table), size=2, replace=False)
alpha = np.random.random()
nuevo = X_min[i].copy()
nuevo[continuous_mask] = (X_min[i][continuous_mask] * alpha +
X_min[j][continuous_mask] * (1 - alpha))
nuevo[~continuous_mask] = (X_min[i][~continuous_mask] if alpha > 0.5
else X_min[j][~continuous_mask])
# Redondear columnas que deben ser enteras
nuevo[integer_mask] = np.round(nuevo[integer_mask]).astype(float)
synthetic_X.append(nuevo)
meta_source_indices.append(i if alpha > 0.5 else j)
synthetic_X = np.array(synthetic_X)
Y_synthetic = np.full(n_needed, Y_min[0])
# Metas: copiar de instancias originales (respeta strings)
if domain.metas:
M_synthetic = np.array([minority_table.metas[k]
for k in meta_source_indices])
else:
M_synthetic = None
# -----
# Agregar columna "origen" como meta string
origen_var = StringVariable("origen")
new_metas = list(domain.metas) + [origen_var]
new_domain = Domain(domain.attributes, domain.class_var, new_metas)
# Datos originales: agregar "original" en la columna origen
n_original = len(in_data)
if domain.metas:

```

```

metas_originales = np.hstack([
in_data.metas,
np.array(["original"] * n_original)
])
else:
metas_originales = np.array(["original"] * n_original)
original_table = Table.from_numpy(
new_domain,
in_data.X,
in_data.Y,
metas_originales
)
# Datos sintéticos: agregar "sintetico" en la columna origen
if domain.metas:
metas_sinteticas = np.hstack([
M_synthetic,
np.array(["sintetico"] * n_needed)
])
else:
metas_sinteticas = np.array(["sintetico"] * n_needed)
synthetic_table = Table.from_numpy(
new_domain,
synthetic_X,
Y_synthetic,
metas_sinteticas
)
# Combinar y mezclar
combined = Table.concatenate([original_table, synthetic_table])
indices = np.random.permutation(len(combined))
out_data = combined[indices]
vals_final = np.array([str(inst.get_class()) for inst in out_data])
print(f"FINAL    →    '{clase_mayoritaria}':    {np.sum(vals_final==clase_mayoritaria)}    |
'{clase_minoritaria}': {np.sum(vals_final==clase_minoritaria)} | Total: {len(out_data)}")
print(f"Columna 'origen' agregada: 'original' o 'sintetico'")

```